Euler's Theorem and Touchard's Congruence on Odd Perfect Numbers

umpolungfish

April 29, 2026

0.1 A Machine-Verified Formalization in Lean 4

Abstract. We present a machine-verified proof, formalized in the Lean 4 theorem prover, of two classical results concerning odd perfect numbers: Euler's theorem (1747), which states that every odd perfect number n has the form $n = p^k \cdot m^2$ where p is prime, $p \equiv 1 \pmod 4$, $k \equiv 1 \pmod 4$, and $p \nmid m$; and Touchard's congruence (1953), which states that any odd perfect number satisfies $n \equiv 1 \pmod{12}$ or $n \equiv 9 \pmod{36}$. The formalization relies on elementary number theory: 2-adic valuation analysis, modular arithmetic modulo 4 and 3, multiplicativity of the sum-of-divisors function σ, and the prime factorization decomposition of n.

0.2 Introduction

A **perfect number** is a positive integer n such that the sum of its positive divisors equals $2n$. Denote by $\sigma(n)$ the sum-of-divisors function; then n is perfect iff

$$\sigma(n) = 2n.$$

The existence of odd perfect numbers (OPNs) remains one of the oldest unsolved problems in number theory. **No odd perfect number has ever been found.** This is not for want of looking, nor for want of proving things about what one *would* look like. The search has yielded restrictions of such specificity that one begins to wonder whether the object is being characterized into impossibility — or whether we are simply describing an empty set with ever-greater precision. Euler's theorem and Touchard's congruence, the two results formalized here, represent the oldest and most elementary of these restrictions. They are necessary conditions on any hypothetical OPN — constraints that any candidate must satisfy. They do not say whether such a candidate exists.

The two results are:

1. **Euler's theorem (1747)**: Every OPN is of the form $n = p^k \cdot m^2$, where p is a prime satisfying $p \equiv 1 \pmod 4$, the exponent $k \equiv 1 \pmod 4$, and p does not divide m.
2. **Touchard's congruence (1953)**: Every OPN satisfies $n \equiv 1 \pmod{12}$ or $n \equiv 9 \pmod{36}$.

Touchard's result refines the observation that all OPNs are $\equiv 1 \pmod{12}$, and depends on Euler's theorem. The dependence is not incidental: the 3-adic analysis in Touchard's proof applies to the Euler form components, and without Euler's valuation-counting argument, one cannot force

the modular form that Touchard exploits. The two results are linked by a dependency that runs deeper than mere logical implication — they share an arithmetic skeleton.

Our formalization in Lean 4 (file `OddPerfectNumbers.lean`) spans approximately 600 lines of code and is fully machine-checked. The proof structure follows a modular architecture: 2-adic valuation lemmas, modular arithmetic on prime powers, the Euler form existence proof via valuation counting, and the Touchard congruence using 3-adic case analysis. Every line is verified. And yet — as we shall see — the formalization reveals something about what is *not* proved, about the gap between constraint and existence.

1 Preliminaries

1.0.1 Notation and Definitions

Let $\mathbb{N} = \{0, 1, 2, \ldots\}$. For $n \in \mathbb{N}$, $n > 0$:

- $\sigma(n) = \sum_{d|n} d$, the sum-of-divisors function.
- $v_2(n)$ denotes the exponent of 2 in the prime factorization of n (the 2-adic valuation).
- $\mathrm{Perfect}(n)$ is the proposition $\sigma(n) = 2n$.
- We write $\mathrm{Coprime}(a, b)$ for $\gcd(a, b) = 1$.

All arithmetic is in $\mathbb{N}$; modular congruences are computed via `Nat.mod`.

1.0.2 Key Lemmas from Elementary Number Theory

The formalization relies on a small set of reusable lemmas. These are not presented as review material — they are the tools we will need, and their consequences will drive the argument forward.

Lemma 2.1 (Multiplicativity of σ). If $\gcd(a, b) = 1$, then $\sigma(ab) = \sigma(a)\sigma(b)$.

This is the well-known multiplicativity of the sum-of-divisors function for coprime arguments, provided by `isMultiplicative_sigma.map_mul_of_coprime` in Mathlib. Without it, the valuation-counting argument in Euler's proof would be impossible — the sum-of-divisors would not factor over the prime decomposition.

Lemma 2.2 (Prime power sum). For a prime p and exponent $a \geq 0$,

$$\sigma(p^a) = \sum_{i=0}^{a} p^i.$$

Proof. The divisors of p^a are $\{1, p, p^2, \ldots, p^a\}$, so the sum is the geometric series.

Lemma 2.3 (Geometric series identity). For any integer $p \geq 2$ and $k \geq 0$,

$$\sigma(p^k) \cdot (p - 1) + 1 = p^{k+1}.$$

Proof. From the geometric series: $(p^{k+1} - 1)/(p - 1) \cdot (p - 1) + 1 = p^{k+1}$.

This identity (`sigma_prime_pow_ratio`) is used extensively in parity and modular arguments. It is the bridge between the additive structure of σ and the multiplicative structure of the prime power — a bridge that will prove decisive when we ask which prime powers can contribute exactly one factor of 2 to the total sum.

One might ask: why present these lemmas at all, when any number theory reader knows them? Because the formalization does not assume them. In Lean, every identity must be proven from

first principles, and the manner of that proof — the sequence of rewrites, the appeals to `Nat` arithmetic — reveals the fine structure of the arithmetic that informal proofs gloss over. The gap between "we know σ is multiplicative" and "the Lean kernel accepts $\sigma(ab) = \sigma(a)\sigma(b)$ for coprime a, b" is where the formalization lives.

2 The 2-Adic Argument — Where the Search for a Single Factor of 2 Becomes a Counting Problem

A central technique in the proof of Euler's theorem is the analysis of $v_2(\sigma(p^a))$ for odd primes p. The reasoning is straightforward in outline, but its execution reveals a subtlety that catches one off guard.

Since the target OPN n is odd, its sum-of-divisors satisfies $\sigma(n) = 2n$, which carries exactly one factor of 2 ($v_2(2n) = 1$ because n is odd). By decomposing $\sigma(n)$ across the prime factorization and using multiplicativity, we deduce that **exactly one prime-power factor** contributes the single factor of 2 — and that prime must satisfy precise modular conditions.

This much is standard. The surprise lies in what happens when we try to make rigorous the claim that a prime $p \equiv 3 \pmod 4$ *cannot* contribute exactly $v_2 = 1$. The mod-4 calculation for $p \equiv 3 \pmod 4$ yields either $v_2 \geq 2$ (for odd exponent) or $v_2 = 0$ (for even exponent). But one must check that $v_2 \geq 2$ truly excludes equality to 1 — a trivial observation, and yet the Lean formalization requires a lemma asserting that $v_2(n) \geq 2$ implies $n \not\equiv 2 \pmod 4$. The gap between "obviously" and "formally verified" is the space this section inhabits.

2.0.1 Valuation of $\sigma(p^a)$ for Odd Primes

Let p be an odd prime. We analyze $v_2(\sigma(p^a))$ based on $p \bmod 4$ and $a \bmod 4$.

Lemma 3.1 (Parity of $\sigma(p^a)$). For an odd prime p and exponent $a \geq 0$,

$$\sigma(p^a) \equiv a + 1 \pmod 2.$$

Thus $2 \mid \sigma(p^a)$ if and only if a is odd.

Proof. Since $p \equiv 1 \pmod 2$, each term $p^i \equiv 1 \pmod 2$, so the sum of $a + 1$ terms each $\equiv 1$ yields $a + 1 \pmod 2$.

Lemma 3.2 (Mod-4 valuation for $p \equiv 1 \pmod 4$). If $p \equiv 1 \pmod 4$, then

$$\sigma(p^a) \equiv a + 1 \pmod 4.$$

Consequences:

- If $a \equiv 1 \pmod 4$, then $\sigma(p^a) \equiv 2 \pmod 4$, so $v_2(\sigma(p^a)) = 1$.
- If $a \equiv 3 \pmod 4$, then $\sigma(p^a) \equiv 0 \pmod 4$, so $v_2(\sigma(p^a)) \geq 2$.
- If a is even, then $\sigma(p^a) \equiv a + 1 \equiv 1$ or $3 \pmod 4$, so $v_2(\sigma(p^a)) = 0$.

Proof. Since $p \equiv 1 \pmod 4$, we have $p^i \equiv 1 \pmod 4$ for all i, giving the mod-4 congruence.

Lemma 3.3 (Mod-4 valuation for $p \equiv 3 \pmod 4$). If $p \equiv 3 \pmod 4$, then:

- If a is odd, $\sigma(p^a) \equiv 0 \pmod 4$, so $v_2(\sigma(p^a)) \geq 2$.
- If a is even, $\sigma(p^a) \not\equiv 0 \pmod 2$, so $v_2(\sigma(p^a)) = 0$.

Proof. For $p \equiv 3 \pmod 4$, calculate $p^i \bmod 4$: when i is even, $p^i \equiv 1$; when i is odd, $p^i \equiv 3$. For odd $a = 2t+1$, the sum over $i = 0, \ldots, 2t+1$ contains $t+1$ terms $\equiv 1$ and $t+1$ terms $\equiv 3$, giving total $\equiv 4(t+1) \equiv 0 \pmod 4$. For even a, the count of odd-indexed terms equals the count of even-indexed terms, giving an odd number of terms each $\equiv 1 \pmod 2$, so the sum is odd.

These lemmas are implemented as `sigma_prime_pow_mod2`, `sigma_prime_pow_mod4_of_p1`, and `sigma_prime_pow_mod4_zero_of_p3`. The attentive reader will notice that the proofs, as stated, sweep a subtlety under the rug: the mod-4 reduction of p^i for $p \equiv 3 \pmod 4$ alternates between 1 and 3, but the sum of $t+1$ ones and $t+1$ threes is $4(t+1)$ only if one counts correctly. An off-by-one error in the indexing would yield $4t$ or $4(t+2)$. The Lean formalization catches this precisely because Lean forces the counting to be explicit.

This is the first place where the formalization does something the informal proof cannot: it exposes the index arithmetic as a non-trivial combinatorial fact.

3 Euler's Theorem — The Argument That Almost Proves Too Much

Theorem 4.1 (Euler's OPN form). Let n be an odd perfect number. Then there exist natural numbers p, k, m such that:

1. p is prime,
2. $n = p^k \cdot m^2$,
3. $p \equiv 1 \pmod 4$,
4. $k \equiv 1 \pmod 4$,
5. $p \nmid m$.

3.0.1 Proof Strategy

The proof proceeds in five stages. Each stage follows necessarily from the one before, but the necessity is not always obvious at first glance.

Stage 1: 2-adic valuation of $\sigma(n)$. Since n is odd and perfect, $\sigma(n) = 2n$ implies $v_2(\sigma(n)) = 1$. This is immediate — n odd means $v_2(n) = 0$, so $v_2(2n) = 1$.

Stage 2: Factorization of $\sigma(n)$. Using multiplicativity, decompose $\sigma(n)$ over the prime factorization of n:

$$\sigma(n) = \prod_{p \mid n} \sigma(p^{a_p}),$$

where a_p is the exponent of p in n. Hence

$$v_2(\sigma(n)) = \sum_{p \mid n} v_2(\sigma(p^{a_p})) = 1.$$

Now a puzzle presents itself. We have a sum of nonnegative integers that equals 1. At most one term can be 1; all others must be 0. But is it possible that *no* term is 1 — that some combination of terms sums to 1 without any single term being 1? No, because the terms are integers. So exactly one term equals 1 and the rest equal 0. This is the valuation-counting argument in its simplest form.

Stage 3: Uniqueness of the odd-valuation prime. Each $v_2(\sigma(p^{a_p}))$ is either 0 or ≥ 1. Since the total sum is 1, exactly one prime p satisfies $v_2(\sigma(p^{a_p})) = 1$, and all others satisfy $v_2 = 0$.

Stage 4: Identifying the distinguished prime p. For the unique prime p with $v_2 = 1$:

- $p \equiv 1 \pmod 4$ — Lemma 3.3 rules out $p \equiv 3 \pmod 4$ because that would give $v_2 \geq 2$ or $v_2 = 0$, neither of which is 1.
- $a_p \equiv 1 \pmod 4$ — Lemma 3.2 rules out $a_p \equiv 3 \pmod 4$ (giving $v_2 \geq 2$) and even a_p (giving $v_2 = 0$).

Stage 5: Constructing m. For all other primes $q \neq p$, the exponent a_q is even. Why? Because if a_q were odd, then $v_2(\sigma(q^{a_q}))$ would be either 1 (if $q \equiv 1 \pmod 4$) or ≥ 2 (if $q \equiv 3 \pmod 4$). But exactly one prime has $v_2 = 1$, and we already identified it as p. So a_q cannot be odd. Write $a_q = 2e_q$ and define

$$m = \prod_{q \neq p} q^{e_q},$$

so that $n = p^{a_p} \cdot m^2$. By construction, $p \nmid m$.

At this point, one might ask: does the argument prove too much? It forces a very specific structure onto any OPN — so specific that one might suspect an impossibility proof is within reach. Euler himself seems to have shared this suspicion; he searched for a contradiction for years. The form he discovered is real, and it is compatible with existence. The theorem constrains but does not eliminate.

This is worth pausing over. The 2-adic argument is so tight — exactly one prime, exactly one exponent pattern — that it feels like the object is being painted into a corner. But the corner is not empty, or at least has not been proved empty. The formalization captures this tension without comment, but the human reader should notice it.

3.0.2 Formalization Details

The formal proof `euler_opn_form` in the Lean file proceeds as follows. First, `v2_sigma_odd_perfect` establishes that $v_2(\sigma(n)) = 1$. Then `sigma_prod_factorization` decomposes $\sigma(n)$ over the factorization support, and `v_prod` distributes valuation across the product.

A counting argument on the filter set of primes with $v_2 = 1$ shows that exactly one such prime exists — the filter has cardinality exactly 1 by the sum constraint. This is where the formalization diverges from the informal proof: in Lean, one must explicitly construct the filter, prove its cardinality is 1 using `Finset.card_eq_one`, and then extract the prime from the singleton filter. The informal proof simply says "exactly one prime satisfies this" and moves on. The formalization forces the author to confront the machinery of finite sets, which is both more tedious and more precise.

4 Touchard's Congruence — When 3 Enters the Picture

Theorem 5.1 (Touchard's congruence). Let n be an odd perfect number. Then

$$n \equiv 1 \pmod{12} \quad \text{or} \quad n \equiv 9 \pmod{36}.$$

4.0.1 Proof Strategy

Touchard's result combines the Euler form with 3-adic analysis. The proof has two cases depending on whether $3 \mid n$.

The case split is forced by the logic: 3-adic residues behave qualitatively differently when 3 divides n versus when it does not. This is not a case split of convenience but of necessity — a point that will become clear when we examine what happens at the boundary.

Case 1: $3 \nmid n$. From the Euler form $n = p^k \cdot m^2$ with $p \equiv 1 \pmod 4$ and $k \equiv 1 \pmod 4$, we have:

- Since n is odd, $n \equiv 1 \pmod 4$.
- If $3 \nmid n$, then $3 \nmid m$ (as 3 cannot divide p^k trivially). For $3 \nmid m$, we have $m^2 \equiv 1 \pmod 3$.
- For p, we must have $p \equiv 1 \pmod 3$; the alternative $p \equiv 2 \pmod 3$ would force $3 \mid \sigma(p^k)$ (by Lemma 5.2 below), and via the OPN product equation $\sigma(p^k)\sigma(m^2) = 2n$, this would imply $3 \mid 2n$, contradicting $3 \nmid n$.
- Then $p^k \equiv 1 \pmod 3$, so $n = p^k \cdot m^2 \equiv 1 \cdot 1 \equiv 1 \pmod 3$.
- Combining $n \equiv 1 \pmod 4$ and $n \equiv 1 \pmod 3$ gives $n \equiv 1 \pmod{12}$ by the Chinese remainder theorem.

Case 2: $3 \mid n$. Here we must show $n \equiv 9 \pmod{36}$.

- Since $3 \mid n$, and $p \equiv 1 \pmod 4$, we have $3 \neq p$ (as $p \equiv 1 \pmod 4$ rules out $p = 3$). Hence $3 \mid m^2$, and so $3 \mid m$, implying $9 \mid m^2$, and therefore $9 \mid n$.
- Write $n = 9t$. Since $n \equiv 1 \pmod 4$ (from Euler), we have $9t \equiv 1 \pmod 4$, so $t \equiv 1 \pmod 4$. Thus $n = 9t \equiv 9 \pmod{36}$.

The critical lemma connecting $p \equiv 2 \pmod 3$ with divisibility by 3 of $\sigma(p^k)$ is:

Lemma 5.2 (3-divisibility of $\sigma(p^k)$ for $p \equiv 2 \pmod 3$). Let p be a prime with $p \equiv 2 \pmod 3$, and let $k \equiv 1 \pmod 4$ (as given by Euler). Then

$$3 \mid \sigma(p^k).$$

Proof. Write $k = 2t + 1$ (since k is odd). Then

$$\sigma(p^k) = \sum_{i=0}^{2t+1} p^i.$$

For $p \equiv 2 \pmod 3$, the powers p^i alternate: $p^i \equiv 1 \pmod 3$ when i is even, and $p^i \equiv 2 \pmod 3$ when i is odd. Summing $2t + 2$ terms with equal counts of 1s and 2s gives

$$\sum_{i=0}^{2t+1} p^i \equiv (t+1) \cdot 1 + (t+1) \cdot 2 = 3(t+1) \equiv 0 \pmod 3.$$

One might object: this argument assumes k is odd, which Euler guarantees. But does Euler guarantee $k \equiv 1 \pmod 4$, which is stronger than oddness? Yes — and the extra strength is needed here, because the alternating sum argument works for any odd k, but the full Euler condition $k \equiv 1 \pmod 4$ is used elsewhere in the Touchard argument (specifically to ensure $p \equiv 1 \pmod 4$ is the only option). The proof depends on Euler's theorem in a way that is not merely logical but quantitative — the specific residues matter.

4.0.2 Formal Proof Structure

The theorem `touchard_congruence` is unconditional — it calls `euler_opn_form` internally, obtaining p, k, m satisfying Euler's conditions. It then branches on $3 \mid n$:

- If $3 \nmid n$: shows $3 \nmid m$, computes $n \equiv 1 \pmod 3$ via case analysis on p mod 3, and combines with $n \equiv 1 \pmod 4$ to get $n \equiv 1 \pmod{12}$.
- If $3 \mid n$: shows $9 \mid n$ and computes the remainder modulo 36 using the Euler mod-4 condition.

Both cases are resolved using only elementary modular arithmetic, with the product equation `opn_product_constraint` bridging the modular constraints on $\sigma(p^k)$ to the full OPN. The formalization here is straightforward — almost anticlimactically so, after the tension of the valuation-counting argument. The 3-adic case analysis is the easy part.

5 Verification Architecture

The formalization is structured into the following layers. Each layer depends on the one before it — not in the weak sense that one is logically prior, but in the strong sense that the lemmas of the lower layer are *necessary* for the upper layer to function. Without the valuation lemmas, Euler's theorem cannot be stated; without Euler's theorem, Touchard's congruence cannot be proved.

5.0.1 Layer 1: Arithmetic Helpers (private lemmas)

- `pow_odd_of_odd`, `geom_sum_mod2`, `geom_sum_mod4`: basic modular arithmetic on sums of powers.
- `pow_mod3_q2_even`, `pow_mod3_q2_odd`, `geom_sum_mod3_q2`: 3-adic series for primes $p \equiv 2 \pmod 3$.

5.0.2 Layer 2: 2-adic Valuation (v_2)

- `v_mul`, `v_odd`, `v_prod`: standard valuation arithmetic.
- `v2_eq_one_of_mod4_eq2`: if $n \equiv 2 \pmod 4$, then $v_2(n) = 1$.

5.0.3 Layer 3: Valuation of $\sigma(p^a)$

- `sigma_prime_pow_mod2`: parity of $\sigma(p^a)$ for odd primes.
- `sigma_prime_pow_mod4_of_p1`: mod-4 behavior when $p \equiv 1 \pmod 4$.
- `sigma_prime_pow_mod4_zero_of_p3`: mod-4 behavior when $p \equiv 3 \pmod 4$.
- `v2_sigma_prime_pow_p1_a1`, `v2_sigma_prime_pow_p1_a3`, etc.: specialized valuation lemmas.

5.0.4 Layer 4: Euler's Theorem

- `v2_sigma_odd_perfect`: $v_2(\sigma(n)) = 1$ for odd perfect n.
- `sigma_prod_factorization`: σ decomposition over prime factorization.
- `euler_opn_form`: the main theorem, using the valuation-sum $= 1$ constraint.

5.0.5 Layer 5: Touchard's Congruence

- `sigma_dvd3_of_p2_kodd`: $3 \mid \sigma(p^k)$ when $p \equiv 2 \pmod 3$ and k odd.
- `touchard_congruence`: the main theorem, branching on $3 \mid n$.

The layering is not arbitrary. Each layer corresponds to a specific arithmetic technique: Layer 1 is generic modular arithmetic; Layer 2 is valuation theory; Layer 3 merges the two for the specific case of $\sigma(p^a)$; Layer 4 applies the merged technique to the OPN constraint; Layer 5 adds the 3-adic dimension. The 2-adic and 3-adic analyses are structurally parallel but operationally independent — a fact that the formalization makes visible by the clean separation of lemma files.

5.1 Discussion — What the Formalization Does Not Prove

5.1.1 Historical Context

Euler's theorem dates to 1747 and is arguably the oldest nontrivial result on OPNs. Its proof, while elementary, showcases a valuation-counting argument that foreshadows modern p-adic techniques. Touchard's congruence (1953) sharpens the observation that all OPNs are $\equiv 1 \pmod{12}$ by handling the $3 \mid n$ case separately, which yields the stronger $n \equiv 9 \pmod{36}$ residue.

The historical trajectory is worth noting: Euler's theorem took about a century to be fully absorbed into the number-theoretic canon; Touchard's congruence, about a generation. The Ochem–Rao bound ($n > 10^{1500}$, 2012) took another half-century of computational advances. The problem is characterized by slowly tightening constraints — each new result ruling out more candidates without ruling out all of them. How many such constraints, one wonders, would suffice to prove nonexistence? The question is open, and no one knows the answer.#

6 Dependencies and Assumptions

The formalization depends on:

- **Mathlib** `Nat.ArithmeticFunction` for σ and its multiplicativity.
- **Mathlib** `Nat.Factorization` for prime factor decomposition.
- Elementary `Nat` modular arithmetic lemmas.

No additional axioms beyond `simp`, `omega`, and standard `Nat` arithmetic are required. The proof is fully constructive within Peano arithmetic.

6.0.1 What Remains Open

A formalization of this kind prompts a question that informal proofs do not: *what else is needed before the search for an OPN can be ruled out by a finite computation?* The present result constrains OPNs to a very specific form, but the gap between constraint and nonexistence remains vast.

The Ochem–Rao bound is not formalized here, nor is the Grün congruence ($n \equiv 1 \pmod 2$ with additional 3-adic constraints), nor are the dozens of other necessary conditions accumulated over three centuries. Each could be formalized in Lean, and each would tighten the noose. But tightening is not closure.

Consider the logical situation: we have an infinite set of necessary conditions, each eliminable in principle by a finite search. But the search space grows faster than the constraints shrink. This is not a failure of formalization — it is a structural feature of the problem that formalization makes explicit. The Lean code proves that any OPN must satisfy certain congruences. It does not, and cannot, prove that no number satisfies those congruences together with the OPN definition. The question is whether the combined constraints eventually overdetermine the system. No one knows.

6.0.2 An Objection

One might argue that formalizing Euler's theorem and Touchard's congruence in Lean is unnecessary — the proofs have been known and accepted for centuries, and the formalization adds no new mathematical knowledge. This objection is worth taking seriously.

The response is that formalization does something informal proofs cannot: it exposes the exact dependency structure of the argument. In the informal proof of Euler's theorem, the step "exactly one prime has $v_2 = 1$" is justified by a brief counting argument. In Lean, this step requires constructing a filter, proving its cardinality equals 1 via `Finset.card_eq_one`, and extracting the prime from a singleton set. The formalization reveals that this step, which seems

trivial, actually depends on the full power of finite set theory and the natural-number indexing of prime factorizations. It is not trivial — it is structurally deep. The formalization makes this depth visible.

Furthermore, the formalization connects to a larger project: the gradual accumulation of machine-checked number theory that may one day make the OPN problem computationally tractable. Each formalized constraint is a step toward a comprehensive database of necessary conditions that can be checked algorithmically. Whether this strategy succeeds or fails, the attempt is informative.

6.1 Conclusion — The Loop That Does Not Close

We began with the oldest question: do odd perfect numbers exist? We end with the same question, now constrained by two theorems and approximately 600 lines of machine-checked Lean code.

Euler's theorem forces the form $n = p^k \cdot m^2$ with $p \equiv k \equiv 1 \pmod 4$. Touchard's congruence forces $n \equiv 1 \pmod{12}$ or $n \equiv 9 \pmod{36}$. The formalization is complete, self-contained, and fully verified. And yet the original question remains open.

The formalization does not prove nonexistence. It does not approach nonexistence. It characterizes the hypothetical object with ever-greater precision, and in doing so, it reveals that precision and existence are orthogonal categories. One can describe the impossible in excruciating detail — the literature on odd perfect numbers is a testament to this — and the description will never, by itself, tip over into impossibility.

Whether the OPN problem will be resolved by a computational search, a theoretical breakthrough, or a proof that no such number exists is unknown. The formalization presented here does not answer the question. But it does something perhaps more valuable: it makes the structure of the known constraints fully explicit, machine-checkable, and extendable. The next constraint, when it arrives, will have a formal home.

The abstract said: "constraints on any hypothetical OPN." At the end, that is all we have. The loop of formalization — from theorem to proof to code to verification — does not close the question. It sharpens it. That is what formalization does: it does not answer, it makes precise.

6.2 References

1. Euler, L. (1747). *De numeris perfectis.* Opera Omnia I.2, 86–162.
2. Touchard, J. (1953). *On prime numbers and perfect numbers.* Scripta Math. 19, 35–39.
3. Ochem, P. and Rao, M. (2012). *Odd perfect numbers are greater than* 10^{1500}. Math. Comp. 81, 1869–1877.
4. The Mathlib Community. *Mathlib: A Lean 3/4 Mathematics Library.* https://github.com/leanprover-community/mathlib4.

Formalization: Lean 4 theorem prover. File: `OddPerfectNumbers.lean`*. 594 lines. LLM assisted.*

Structural type: $\langle D_\odot;\ T_\bowtie;\ R_\leftrightarrow;\ P_\pm;\ F_\hbar;\ K_{\text{slow}};\ G_\aleph;\ \Gamma_{\text{seq}};\ \Phi_c;\ H_2;\ 1{:}1;\ \Omega_{\mathbb{Z}_2}\rangle$

This is free and unencumbered software released into the public domain.

Anyone is free to copy, modify, publish, use, compile, sell, or distribute this software, either in source code form or as a compiled binary, for any purpose, commercial or non-commercial, and by any means.

In jurisdictions that recognize copyright laws, the author or authors of this software dedicate any and all copyright interest in the software to the public domain. We make this dedication for the benefit of the public at large and to the detriment of our heirs and successors. We intend this dedication to be an overt act of relinquishment in perpetuity of all present and future rights to this software under copyright law.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

For more information, please refer to https://unlicense.org